Problem Analysis

Requirements and business rules

Customers may register, sign in and out, search dated flights, inspect availability, book one to four passengers, retrieve their own reservations and cancel them. Administrators maintain airports, aircraft, routes, departures, fares and reservations. Roles are checked on the server. A public registration request always creates a customer; it cannot grant administrator privileges.

Each booking belongs to one customer and exactly one FlightInstance. Flight is a reusable route and flight number; FlightInstance is a particular departure, aircraft, arrival and fare. There is no multi-segment itinerary. Passenger records contain fictional names. The BookingPassenger junction makes group bookings explicit. SeatAssignment identifies a passenger's seat on one departure and retains released allocations as history.

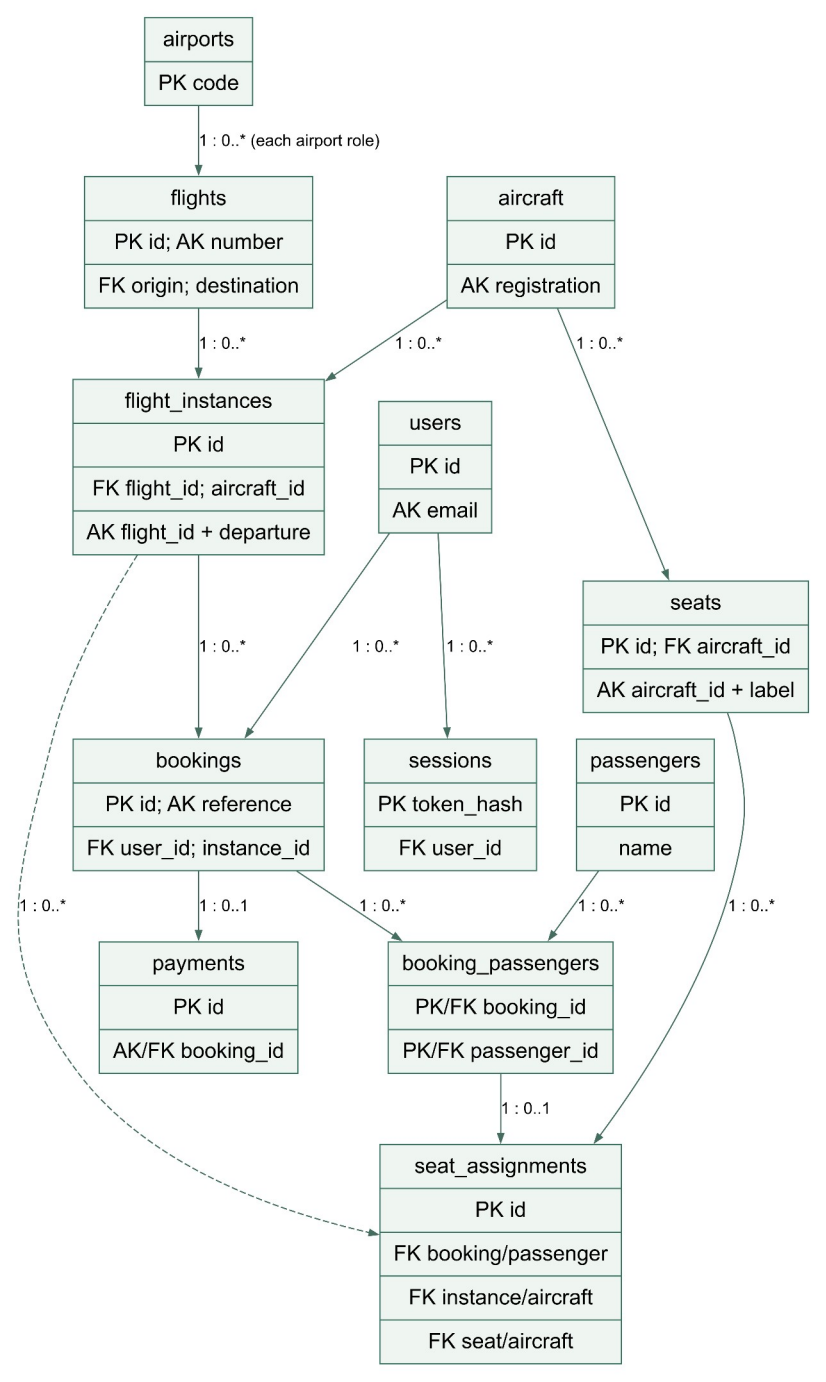
Rule	Implementation
Capacity	An aircraft owns a fixed set of Seat rows. An active seat can be assigned once per departure.
Fare	Positive integer cents. Total = saved unit fare × passenger count. Clients cannot set totals.
Booking states	Confirmed -> cancelled. No reopening; create a new booking.
Departure states	Scheduled -> cancelled. Cancelling a departure cancels its bookings atomically.
Payment	One simulated record per successful booking; cancellation changes it to voided.
Availability	Only future scheduled departures can be booked. Aircraft schedules cannot overlap.
Ownership	Every customer lookup and cancellation filters by authenticated user ID.

Transactions and constraints

Creation locks the dated departure before inserting all related rows in a single transaction. A generated active_seat value equals seat_id only while released_at is NULL. UNIQUE(instance_id, active_seat) prevents active duplicates while permitting multiple historical released rows. Composite foreign keys ensure the seat's aircraft matches the departure and the allocation's departure matches the booking. Rollback removes every insert if any passenger fails.

Entity Relationship Design

The diagram specifies the implemented ER model. SQL permits zero children; application transactions require 1 to 4 passengers and one payment per booking. PK = primary key; AK = alternate key; FK = foreign key. Each child FK is mandatory unless explicitly nullable. All twelve implemented tables are included.



Additional composite FKs: seat_assignments(booking_id, passenger_id) -> booking_passengers; (booking_id, instance_id) -> bookings; (instance_id, aircraft_id) -> flight_instances; (seat_id, aircraft_id) -> seats. Each assignment requires exactly one matching parent in all four relationships.

Normalization and Dependencies

Unnormalized starting point

A spreadsheet reservation might store Reference, CustomerEmail, CustomerName, FlightNumber, OriginCity, Departure, AircraftRegistration, SeatList, PassengerNameList, Fare, Total and PaymentStatus in one row. Lists such as “1A, 1B” violate atomicity. Repeating customer, aircraft and airport descriptions creates update anomalies; deleting the last reservation could also delete the only stored airport information.

First normal form

Replace repeating passenger and seat lists with one row per booking and passenger. Each cell now holds one value. A tentative relation keyed by (BookingReference, PassengerID) still repeats booking fare, customer and flight details for every passenger. Inserting a route without a booking remains difficult, and updating a customer email in one row can leave conflicting copies.

Second normal form

Separate attributes dependent on only part of the composite key. BookingReference determines customer ID, departure ID, unit fare, total and status. PassengerID determines passenger name. BookingPassenger contains only the composite booking/passenger association. Each fact is now attached to its full key. SeatAssignment relates that pair to a seat and dated departure.

Third normal form

Remove transitive dependencies: UserID -> Email, Name, PasswordHash, Role; AirportCode -> Name, City, Timezone; AircraftID -> Registration, Model; FlightID -> Number, Origin, Destination; InstanceID -> FlightID, AircraftID, Departure, Arrival, CurrentFare, Status. Keeping these facts in separate tables avoids copying airport names or aircraft models into each reservation.

Dependency	Design consequence
(aircraft_id, label) -> seat_id	Seat labels are unique within an aircraft, not globally.
(flight_id, departure) -> instance_id	The same route can depart on many dates.
booking_id -> reference, user_id, instance_id, saved fare, status	Booking keeps its sale-time fare even after schedule fare edits.
(booking_id, passenger_id) -> seat assignment	A booked passenger has one seat for this single-segment itinerary.
booking_id -> payment amount and status	Unique payment booking_id implements at most one simulated payment.

Controlled redundancy: booking total and unit fare are sale-time financial snapshots; payment amount is a reconciliation record. Aircraft and instance IDs in assignments support composite integrity constraints. The generated active_seat is derived rather than manually updated. Data-quality queries check these deliberate repetitions.

Table Specifications Part One

MySQL 8.4 types. U means UNSIGNED. Null Y/N indicates SQL nullability. All identifiers are auto-increment unless a natural or composite key is stated. Exact DDL is schema.sql.

Table and column	Type	Null	Key or reference
aircraft.id	int U	N	PK
aircraft.registration	varchar(12)	N	AK
aircraft.model	varchar(80)	N	-
airports.code	char(3)	N	PK
airports.name	varchar(100)	N	-
airports.city	varchar(80)	N	-
airports.timezone	varchar(60)	N	-
flight_instances.id	bigint U	N	PK
flight_instances.flight_id	int U	N	indexed FK flights.id
flight_instances.aircraft_id	int U	N	indexed FK aircraft.id
flight_instances.departure	datetime	N	indexed
flight_instances.arrival	datetime	N	-
flight_instances.fare_cents	int U	N	-
flight_instances.status	ENUM status	N	-
flights.id	int U	N	PK
flights.number	varchar(10)	N	AK
flights.origin	char(3)	N	indexed FK airports.code
flights.destination	char(3)	N	indexed FK airports.code
seats.id	int U	N	PK
seats.aircraft_id	int U	N	indexed FK aircraft.id
seats.label	varchar(4)	N	-
sessions.token_hash	char(64)	N	PK
sessions.user_id	bigint U	N	indexed FK users.id
sessions.csrf_token	char(48)	N	-
sessions.expires_at	datetime	N	indexed
users.id	bigint U	N	PK
users.name	varchar(80)	N	-
users.email	varchar(200)	N	AK
users.password_hash	varchar(200)	N	-
users.role	ENUM role	N	-
users.created_at	timestamp	N	-

Alternate composite keys: seats(aircraft_id,label), seats(id,aircraft_id), flight_instances(flight_id,departure), flight_instances(id,aircraft_id). Role ENUM: customer/admin. Departure status ENUM: scheduled/cancelled. Check constraints reject equal origin/destination, nonpositive fares and arrival before or equal to departure.

Table Specifications Part Two

Table and column	Type	Null	Key or reference
booking_passengers.booking_id	bigint U	N	PK FK bookings.id
booking_passengers.passenger_id	bigint U	N	PK FK passengers.id
bookings.id	bigint U	N	PK
bookings.reference	char(16)	N	AK
bookings.user_id	bigint U	N	indexed FK users.id
bookings.instance_id	bigint U	N	indexed FK flight_instances.id
bookings.status	ENUM status	N	-
bookings.unit_fare_cents	int U	N	-
bookings.total_cents	int U	N	-
bookings.created_at	datetime	N	-
bookings.cancelled_at	datetime	Y	-
passengers.id	bigint U	N	PK
passengers.name	varchar(80)	N	-
payments.id	bigint U	N	PK
payments.booking_id	bigint U	N	AK FK bookings.id
payments.amount_cents	int U	N	-
payments.status	ENUM payment	N	-
seat_assignments.id	bigint U	N	PK
seat_assignments.booking_id	bigint U	N	indexed composite FK
seat_assignments.passenger_id	bigint U	N	composite FK
seat_assignments.instance_id	bigint U	N	indexed composite FK
seat_assignments.aircraft_id	int U	N	composite FK
seat_assignments.seat_id	int U	N	indexed composite FK
seat_assignments.released_at	datetime	Y	-
seat_assignments.active_seat	int U	Y	-

Booking status ENUM: confirmed/cancelled. Payment status ENUM: simulated/voided. active_seat is a stored generated INT U; it is NULL for released allocations. UNIQUE(instance_id,active_seat) and UNIQUE(booking_id,passenger_id) enforce inventory and passenger allocation rules. bookings(id,instance_id) is a composite alternate key used by an FK.

Indexes and referential actions

Alternate candidate keys identify rows: users.email, aircraft.registration, flights.number, bookings.reference and payments.booking_id. Secondary indexes accelerate access without defining identity: sessions(expires_at), bookings(user_id,created_at), flight_instances(departure,status). MySQL also creates indexes needed by foreign keys; these are distinct concepts. All FKs use RESTRICT/NO ACTION on deletion and update except sessions.user_id, which cascades on user deletion. Parent records with history are preserved. Cancellation changes status instead of deleting business records. Composite FKs listed on page 2 all use RESTRICT. Enrollment, passenger-count and exactly-one-payment rules are enforced by application transactions; the ER diagram distinguishes those rules from SQL optionality.